P_POO – Shoot Em Up



Gillian L'Horset – CIN2B
Vennes
32 periodes
Xavier Carrel

Table des matières

1 SPÉCIFICATIONS 3

1.1 INTRODUCTION 3
1.2 MATÉRIEL ET LOGICIELS À DISPOSITION 3
1.3 CAHIER DES CHARGES 3
1.3.1 Fonctionnalités requises (du point de vue de l'utilisateur) 3
1.3.2 Contraintes 3

2 ANALYSE 3

2.1 PLANIFICATION 3
2.2 ANALYSE FONCTIONNELLE 4
2.3 USER STORIES 4
2.3.1 Déplacement 4
2.3.2 Obstacle 4
2.3.3 Point de vie 5
2.3.4 Types d'ennemis 5
2.3.5 Tirs 5
2.3.6 Système de dégât 5
2.3.7 Tir du joueur 5
2.4 DIAGRAMME UML 5
2.4.1 Enemy 6
2.4.2 Shoot 6
2.4.3 Ship 6
2.4.4 AirSpace 6
2.4.5 Autre 6

3 RÉALISATION 7

3.1 DOSSIER DE RÉALISATION 7
3.1.1 Gameplay 7
3.1.2 Le joueur 7
3.1.3 Tirs 7
3.1.4 Ennemis 7
3.1.5 Points de vie 8
3.2 RÉALISATION TECHNIQUE 8
3.2.1 Ship 8
3.2.2 Ennemis 8
3.2.3 Shoot 8
3.2.4 AirSpace 9

4 CONCLUSION 9

4.1 BILAN DES FONCTIONNALITÉS DEMANDÉES 9
4.2 BILAN DE LA PLANIFICATION 9
4.3 BILAN PERSONNEL 9

5 DIVERS 9

5.1 JOURNAL DE TRAVAIL 9
5.2 WEBOGRAPHIE 9

6 ANNEXES 9

1 SPÉCIFICATIONS

1.1 Introduction

Le but de ce projet est d'appliquer les apprentissages concernant l'orienté objet en c#.

Le cœur du projet est de créer un jeu vidéo ou un joueur sera dans un environnement ou lui et les ennemis pourrons se tirer dessus. Le but pour le joueur est de progresser dans le jeu pour atteindre le boss final, le tuer et remporter la partie.

Le projet à pour but d'être une manière d'apprendre, le développement orienté objet, de manière ludique. Cela permet de stimuler la créativité, de motiver et de s'amuser tout en apprenant du code.

1.2 Matériel et logiciels à disposition

Le jeu tournera sur Windows Form et se basera sur la version fournie par le chef de projet, nommé "scrolling".

1.3 Cahier des charges

1.3.1 Fonctionnalités requises (du point de vue de l'utilisateur)

La grande majorité des fonctionnalités demander dans le cahier des charges sont présentent dans le programme. La seul qui n'est pas respecter, est le point a du point 7.3 du cahier des charges, le fait que « minimum 5 obstacles […] être détruit » à été remplacer, avec l'accord du chef de projet, par un sol qui se déplace de droite à gauche.

1.3.2 Contraintes

La contrainte principale est que le projet est réalisé sur Windows Forms, cela fait que le moteur est peu optimisé pour du jeu, très rigide et ne contiens pas d'aide au développement direct.

Le jeu devant être codé en c# orienté objet, cela rajoute une difficulté d'apprentissage de cet orienté objet.

2 ANALYSE

2.1 Planification

Semaine 3 :
Création de l'environnement de jeu.

Semaine 4 :
Création du joueur et des déplacements.

Semaine 5 :
Déplacement

Semaine 6 :
Tirs et intégration des points de vie.

Semaine 7 - 8
Création des ennemis et leur comportement.

2.2 Analyse fonctionnelle

L'analyse est relativement simple, un joueur (triangle), des ennemis, (rond) des obstacles (rectangles) et des tirs. Il s'agit d'un résumer du gameplay, le joueur et les ennemis peuvent se déplacer, les ennemis à l'aide de comportement préconfigurer. Le sol défile de droite à gauche, pour donner une impression d'avancer. Il y a également des cœurs en haut à gauche pour représenter que le joueur à de la vie



2.3 User stories

2.3.1 Déplacement

Le système de déplacement est très simple, par exemple si le joueur appuie sur D il va à droite, sauf s'il est déjà au bord de l'écran.
https://github.com/Gillian-LHorset/Projet320-ShootEmUp/issues/1

2.3.2 Obstacle

Pour les obstacles, il y a un sol qui est générer aléatoirement et qui défile vers la droite. Le joueur n'est pas en capacité de descendre dedans et s'il le touche (ex il est au niveau du sol et le sol monte et donc le touche) il prend un point de dégât. Ceci est représenter de manière grossière dans l'analyse fonctionnel.
https://github.com/Gillian-LHorset/Projet320-ShootEmUp/issues/2

2.3.3 Point de vie

Le joueur dispose de points de vie, il peut les voir graphiquement, ses points de vie restant, ils sont affiché en permanence sur l'écran. Il à la capacité de se soigné. Les dégâts reçu par le joueur, par les ennemis ou les obstacles, lui donne un temps d'invincibilité.
https://github.com/Gillian-LHorset/Projet320-ShootEmUp/issues/3

2.3.4 Types d'ennemis

Il y a des ennemis qui ont différente caractéristique, de manière générale, les ennemis se déplace de haut en bas, et tir sur le joueur.
https://github.com/Gillian-LHorset/Projet320-ShootEmUp/issues/4

2.3.5 Tirs

Les tirs sont une mécanique centrale du jeu, le joueur et les ennemis tirs pour infliger des dégâts. Le joueur doit les éviter pour ne pas perdre de vie et mourir, mais les ennemis ne cherchent pas forcément à les éviter. Il y a un missile qui inflige plus de dégât et qui peut être lâché à la mort des ennemis.
https://github.com/Gillian-LHorset/Projet320-ShootEmUp/issues/5

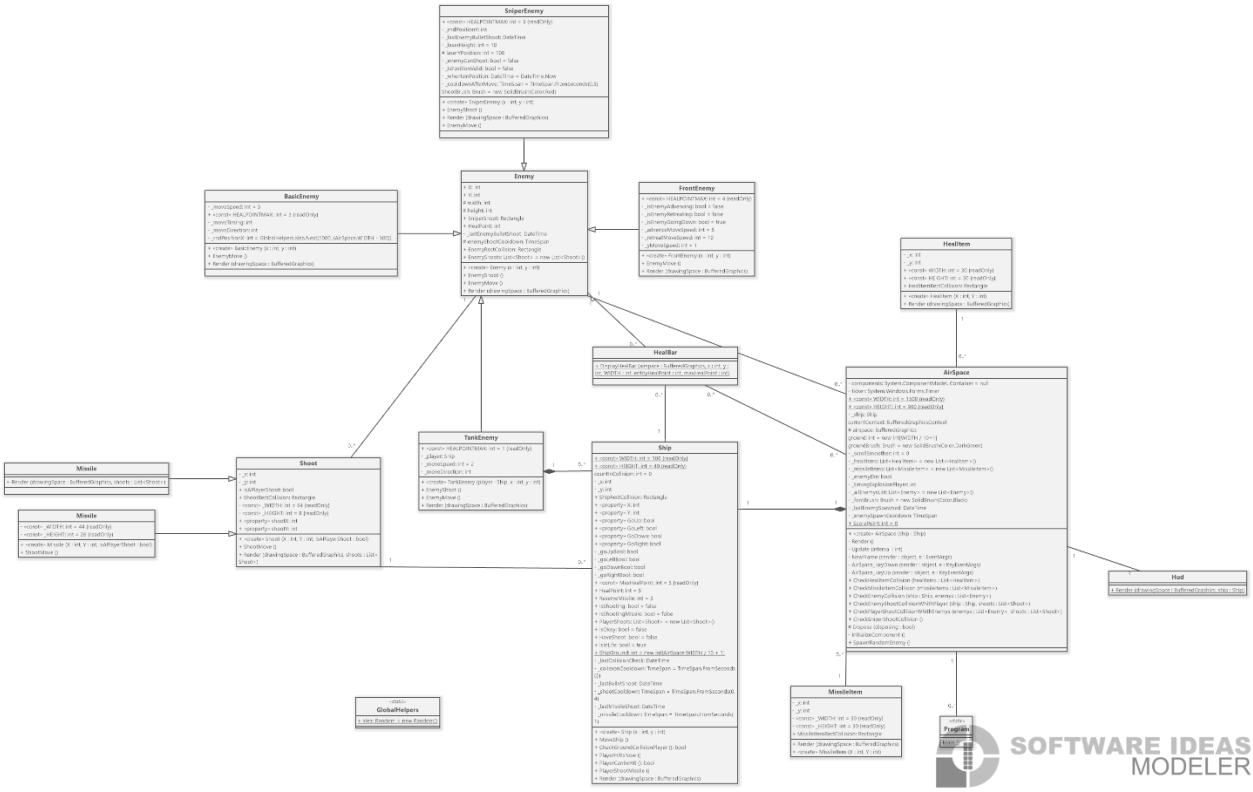
2.3.6 Système de dégât

Les dégâts font diminuer la vie de l'entité avec laquelle il entre en collision. Ils offrent une durée d'invulnérabilité de 2 secondes au joueur après avoir été touché.
https://github.com/Gillian-LHorset/Projet320-ShootEmUp/issues/6

2.3.7 Tir du joueur

Le joueur à la possibilité de tirer avec la barre espace, les tirs ont un cooldown entre eux.
https://github.com/Gillian-LHorset/Projet320-ShootEmUp/issues/7

2.4 Diagramme UML



Pour plus de lisibilité, consulter le fichier "I-P_POO-gillhorset-DiagrammeUML.png" fourni dans la release.
Ce diagramme indique toutes les variables et méthode utiliser dans le projet.

2.4.1 Enemy

Les différents types d'ennemis hérite de la classe Enemy, cela permet d'avoir des variables et méthode en commun. La classe Enemy contient SniperShoot, qui est utiliser pour les collisions du tir du type d'ennemi sniper, il est présent ici pour pouvoir être appelé dans AirSpace pour vérifier les collisions, mais s'il serait uniquement sur le sniper, il ne serait pas possible de l'appeler.
La classe parent contient un Rectangle nommé EnemyRectCollision, il est là pour pouvoir vérifier les collisions avec le joueur ou ses tirs. Elle contient également une méthode de shoot par défaut et des mouvements qui ont pour but d'être override pour que les différentes classes puissent agir différemment. Elle contient des variables DateTime et TimeSpan, qui sont là pour les cooldown des tirs de ennemis.

Le FrontEnemy contient beaucoup de variable pour savoir dans quelle direction bouger.

BasicEnemy, contient des variables pour savoir quand se déplacer (_moveTiming), dans quelle direction se déplacer (_moveDirection) et quelle position horizontale être (_rndPositionX).

SniperEnemy lui a beaucoup de variables pour ses tirs, connaitre leurs taille(_laserHeight), connaitre sa position (_laserYPosition). Il a aussi des variables pour se positionner aléatoirement sur l'axe verticale, savoir quand il est en position (_whenIsInPosition), le temps qu'il lui faut pour tirer après s'être déplacer (_cooldownAfterMove) et savoir s'il est en position pour tirer (_enemyCanShoot).

TankEnemy, lui est lié au joueur, il a besoin de sa position pour se déplacer vers sa direction.

2.4.2 Shoot

Shoot possède un rectangle pour vérifier les collisions avec les entités.
Missile est une classe qui hérite de Shoot, qui a comme différence sa taille et sa vitesse de mouvement.

2.4.3 Ship

Ship est une classe majeure du programme, il contient le vaisseau que le joueur contrôle. Il a des points de vie (HealPoint), des points de vie maximum (MaxHealPoint) qui est en constant, car elle n'a pas besoin d'être changé. Il à une réserve de missile (ReserveMissile) qui indique combien de missile le joueur peut encore tirer. Il a beaucoup de variable pour savoir quand tirer IsShooting, IsShootingMissile, HaveShoot, _lastBulletShoot, _ShootCooldown, _lastMissileShoot et _missileCooldown qui ont un usage spécifique pour chacun. Il a une variable pour savoir si le Joueur est en vie (IsInLife), il a un tableau qui contient les différentes hauteurs du sol (ShipGround) qui va aller récupérer dans AirSpace pour savoir s'il va toucher le sol, qui est utiliser dans la méthode CheckGroundCollisionPlayer. Il a quelques méthodes pour les dégâts reçu, PlayerHitIsNow indique que le joueur vient d'être touché et est utiliser dans PlayerCanBeHit pour savoir si le joueur peut à nouveau être touché.

2.4.4 AirSpace

La variable _ship est simplement la variable qui contient le joueur. Le tableau ground contiens les hauteurs des différents blocs du sol. Les listes _healItems, _missileItems et _allEnemyList sont simplement des listes pouvant contenir les différents objets qui compose le gameplay du jeu. EnemyDie est utiliser dans les méthodes CheckEnemyCollision et CheckMissileCollision pour déterminer si l'ennemi est mort. _timingExplosionPlayer est simplement une variable qui est incrémenter lorsque le joueur meurt pour changer la frame de son animation de mort. Les variables _lastEnemySpawned et _enemySpawnCooldown son utiliser pour déterminer le moment auquel le prochain ennemi va apparaitre et dans combien de temps il va apparaitre. Toutes les méthodes Check… sont utiliser pour vérifier les collisions entre les objets du programme. Les méthodes …KeyDown et …KeyUp sont là pour vérifier lorsque le joueur appuie et relâche une touche du clavier.

2.4.5 Autre

La plupart des classes possède des méthodes Render, elles servent simplement à s'afficher sur l'écran. Certaines possède des variables …RectCollision, elles sont là pour être utiliser pour vérifier quand deux objets se touche.

3 RÉALISATION

3.1 Dossier de Réalisation

Le jeu se base sur quelques grands principes : Le joueur et ses déplacements, les tirs, les ennemis et les points de vie.

3.1.1 Gameplay

Le gameplay du jeu est relativement simple, des ennemis apparaissent et le joueur doit tirer dessus pour les éliminés et gagner des points (100 par ennemi). Une fois arriver à 2000 points le joueur gagne la partie. Les ennemis
tirs également et si la vie du joueur atteint 0 il a perdu la partie.

3.1.2 Le joueur

Le joueur est représenté sur le AirSpace par un vaisseau que le joueur peut déplacer à l'aide des touche W, A, S et D. Le joueur est bloqué lors qu'il essaie de sortir de l'écran ou de descendre en dessous du sol.

3.1.3 Tirs

Le jeu contient plusieurs types de tir, le joueur peut tirer des tirs normaux à l'infinie et des tirs de missiles, qui eux sont en nombre limité. Au début du jeu le joueur possède 3 missiles et lors qu'il en tire un, il est retiré de sa réserve. En tuant des ennemis, il a une chance sur quatre que l'ennemi laisse tomber un objet missile ramassable par le joueur pour incrémenter sa réserve. Les missiles, s'ils touchent l'ennemi, lui retire trois points de vie contre un pour les tirs normaux. Les tirs basiques ont un temps de recharge de 0.4 seconde.

Les ennemis de types "basic" et "front" ont des tirs qui parte en ligne droite de droite à gauche et disparaissent s'ils quittent l'écran ou touche le joueur.

Les ennemis de type "sniper" tirent un rayon laser qui prends toute la largeur de AirSpace instantanément après leur tir.

Les ennemis de type "tank" ne tirent pas.

3.1.4 Ennemis

Il existe 4 types d'ennemis,
Les ennemis de type "basic" bouge aléatoirement de haut en bas et reste sur une position latérale qui leur est propre et défini aléatoirement. Ils ne tirs que des tirs basiques.

Les ennemis de type "front" avance lentement vers la gauche de l'écran, en tirant des tirs basiques, puis retourne rapidement à droite de l'écran. Il se déplace légèrement de haut en bas puis remonte après avoir touché le sol.

Les ennemis de type "sniper" choisissent aléatoirement une position puis s'y déplace et s'y arrête avant de tirer, puis recommence. Ils tirent des lasers qui font instantanément toute la largeur de l'écran.

Les ennemis de type "tank" ne tire pas et possède uniquement un point de vie. En revanche, il n'est pas sensible aux tirs basiques et ne peux être vaincu uniquement à l'aide de missile.

Les déplacements des différents ennemis ne leur permettent pas, ni de sortir du AirSpace, ni de passer sous le sol du niveau.

Au lancement du jeu, il y a 10 ennemis à l'écran, et il en apparait toutes les 4 à 7 secondes.

3.1.5 Points de vie

Chaque entité, que se soit joueur ou ennemi, possède des points de vie. Ces points de vie diminuent, sauf exception, au contacte des projectiles. Alors que les tirs basiques font un point de dégât, les missiles en font trois. Le joueur à la possibilité de se soigner en récupérant des objets de soin qui peuvent être lâché par les ennemis à leur mort.
Lors que les points de vie arrivent à zéro, un ennemi meurt et est donc retirer du jeu. Si les points de vie du joueur arrivent à zéro, la condition de défaite est remplie, donc le joueur explose et un message de défaite apparait.

3.2 Réalisation technique

Mon projet s'articule autour de différents objets qui interagissent entre eux dans AirSpace.

3.2.1 Ship

Le ship est le joueur. La classe possède des attributs classiques, position, taille, direction. Il possède une liste de shoot qui lui est propre, cela rend le code plus facile pour tirer, car on peut juste ajouter dans cette classe et le render dans le AirSpace.

Lors que le joueur tir, on va appliquer un cooldown. On définit le cooldown, on regarde si la dernière fois qu'il à tirer est supérieur au cooldown, si oui on note le dernier tir à maintenant.

Le niveau ayant un sol qui défile, le joueur ne doit pas pouvoir descendre en dessous de se sol. La difficulté est que la hauteur du sol est générée aléatoirement, pour résoudre le problème, on va récupérer le tableau qui contiens les valeurs de hauteur du sol depuis AirSpace, le faire passer dans la classe Ship au travers d'un tableau ShipGround. De là on peut définir que si la hauteur du vaisseau plus sa taille est inferieur à la hauteur du sol, il ne peut pas descendre. De là on peut en déduire que si la condition est fausse, il est dans le sol et on peut donc lui appliquer des dégâts.

Pour éviter que le joueur perde toute sa vie à la moindre collision, on ajoute un cooldown et un effet de clignotement du modèle pour avoir un retour visuel pour l'utilisateur.

3.2.2 Ennemis

Les ennemis à des tirs basique, ils ont de la vie, des tirs (pour la plupart) avec un cooldown et un déplacement qui leur est propre.

Le basic enemy à des tirs simples, il se déplace verticalement durant environ une seconde puis change de direction aléatoirement selon un random qui fait qu'il a une chance sur deux de monter ou descendre. On vérifie que si sa position plus son futur déplacement ne sort pas de l'écran, sinon on bloque son mouvement.

Le front enemy à des tirs basiques, il se déplace, selon une force donnée, vers la gauche, puis lorsque que sa position est plus petite que 200 on lui donne l'ordre de reculer avec une force deux fois supérieur à la force d'avance.

Pour le tank enemy, dans son constructeur, on lui donne une instance du joueur (ship) pour pouvoir utiliser sa position. Si la position X de l'ennemi est inferieure à la position X du joueur, on réduit sa position, pareille pour Y.

Le sniper enemy se déplace uniquement verticalement, il va tirer, choisir une nouvelle position à l'aide d'un random, s'y déplacer, un cooldown va le maintenir en place quelques instants, puis exécuter la méthode de tir.

3.2.3 Shoot

Shoot est une classe plutôt simple, le tir a une variable à sa création qui définit s'il est tiré par le joueur ou un ennemi. L'objet shoot va se déplacer, vers la droite s'il est tiré par le joueur, et vers la gauche s'il est tiré par un ennemi.
Tout l'intérêt du shoot est de pouvoir être créer par les autres classes de manière facile pour tirer.

3.2.4 AirSpace

Le AirSpace contient, à peu près, toutes les listes et objets du jeu.
Il contient les listes des items, des ennemis, la liste des tirs ennemis et le joueur. Cela est fait pour pouvoir créer très simplement des méthodes en utilisant les objets présents de dedans. La grande majorité des méthodes de check de collision sont présente dedans, comme il contient déjà toutes les listes cela rends le tout beaucoup plus facile.

4 CONCLUSION

4.1 Bilan des fonctionnalités demandées

Toutes les fonctionnalités indiquer par les user stories au début du projet ont été réaliser. Le projet contient également des fonctionnalités qui ont été ajouter au cours du projet et qui ne figurait pas dans les user stories de départ.

4.2 Bilan de la planification

La planification n'a rien à voir avec la réalité, de très grosses difficultés ont été rencontrer au début du projet qui ont grandement ralenti le tout.
Les 3 premières semaines ont été consacré à l'apprentissage de la base du développement orienté objet.
La 4ème semaine à été consacré à la création du joueur.
Les semaine 5 à 7 ont été consacrés au déplacement du joueur, donc un gros retard sur le programme.
La dernière semaine à été dédier uniquement à la documentation.

Le projet ayant pris un énorme retard, j'ai passé les deux semaines de vacances scolaires à développer le projet, cela fait que ces semaines ont permis de finaliser les déplacements du joueur, créer le système de tirs, créer un système de points de vie et la création des ennemis et de leurs comportements. Les fonctionnalités demander ont donc été rendu-

4.3 Bilan personnel

Le projet à été très compliquer personnellement. N'ayant jamais développer en orienté objet avant, le module à été une découverte totale pour moi. Ayant eu de grosses difficultés dès le début du projet, j'ai très peu progressé dans la création du projet. Voyant le temps passer et le projet stagné, j'ai décider de passer les deux semaines de vacances scolaires à travailler sur le projet, et ça a porté ses fruits. Je sais désormais coder en orienté objet et je considère le projet comme m'ayant permis de mieux comprendre la manière de développer et d'apprendre dans le domaine du développement.

5 DIVERS

5.1 Journal de travail

Voir fichier nommé "T-P_POO-gillhorset-JournalTravail.pdf"

5.2 Webographie

Les différents assets utilisés pour le jeu, viennent du jeu "Gradius" sortie sur la console de jeu NES en 1988. Le gameplay s'en inspire également.

https://learn.microsoft.com/en-us/dotnet/csharp/
https://www.w3schools.com/cs

6 ANNEXES

Pour le diagramme UML voir le fichier "I-P_POO-gillhorset-DiagrammeUML.png"